

CSCI-304-02 Computer Organization: Assignment 3

October 27, 2023

Due: 11:59pm 10/20/2023

Submission: Please submit a `.c` file (C source code), a `.txt` file (explain compile details) and a PDF file (your answers to the questions) on Blackboard.

Show your work for full credit. All submission must be completely your own work.

1. [50 points] **Project: Simple Cipher in C**

In this project, you are tasked with creating a C program to encode a provided message. The detailed requirements are as follows:

- **Encoding Scheme:**

- Reverse each “word” in the given phrase. A “word” is defined as a sequence of alphabetic characters separated by one or more non-alphabetic characters.
- Maintain the position of all non-alphabetic characters. As a result, the length of the output string should match the length of the input string.

- **Handling Underscores:**

- The input message will use underscores (`'_'`) instead of spaces to facilitate reading it as a single text value.
- In your encoded output, convert all underscores back to spaces.

- **Case Handling:**

- Ensure all alphabetic characters in the output are in uppercase.

- **Program Termination:**

- Each input should end with a newline.
- Users can exit the program by entering the word “quit”.

Example inputs -> outputs:

- War_eagle -> RAW ELGAE
- Reading_records_of_variable_length? -> GNIDAER SDROCER FO ELBAIRAV HTGNEL?
- Have_fun._Doing_this_lab_:) -> EVAH NUF. GNIOD SIHT BAL :)
- Words%end*with^non-alpha...characters!!! -> SDROW%DNE*HTIW^NON-AHPLA...SRETCARAHC!!!

CONSTRAINTS:

- The length of the input message to be encoded will not exceed 100 characters, including the new line character.
- Use pointers to manipulate the location of each character in the array(s)
- All code should be submitted in a single source file named *simple_cipher.c*. Please specify the command you use to compile your program either in a *.txt file* or by submitting a *Makefile* along with your solution.

Your program can be compiled with the command:

```
$ gcc simple_cipher.c -o simple_cipher
```

..and run with the command to test with the sample input file on the blackboard:

```
$ ./simple_cipher < test2.txt
```

or with the command (to save the output):

```
$ ./simple_cipher < test2.txt > mytestout2.txt
```

Compare results with sample results on the class webpage using the command:

```
$ diff mytestout2.txt testout2.txt
```



2. [6 points] Consider the following C code. What is the value of variable s after the for loop? Explain how you get the value of s .

```
int i; /* 32 bit integer */
unsigned int s=1; /* 32 bit unsigned integer */
for (i = -1 ; i < 0; i--)
{
    s = s+1;
}
```

i is a 32 bit signed integer, which can be from $-(2^{31})$ to $(2^{31} - 1)$
(-2,147,483,648 to 2,147,483,647)

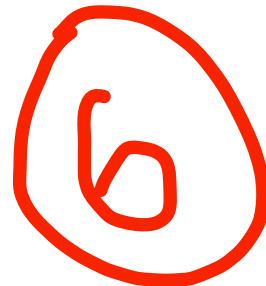
s is a 32 bit unsigned integer, which can be from 0 to $2^{32} - 1$ (0 to 4,294,967,296)

s starts as 1, and i starts as -1

The loop will run from $i = -1$ to $i = -2,147,483,648$, terminating when i overflows to zero

Therefore, the loop runs, incrementing s , 2,147,483,648 times

At the end, s will be 2,147,483,649 (1+ 2,147,483,648)



3. [12 points] Show how values are stored in memory using the Sun, IA32, and x86-64 formats, as demonstrated on slide 80 of Lec04_integers_02072023. Assume each value is declared as a `long int` in C. Both Sun and IA32 use 32-bit encoding for `long int`, while x86-64 uses 64-bit encoding.

a. -862

$$-862_{10} = 1100\ 1010\ 0010_2 = (-2048 + 1024 + 128 + 32 + 2)$$

$$1100\ 1010\ 0010_2 = \text{CA}_{16}$$

expand to 32 bits for IA32 and Sun: FFFFFFCA_{16} ,

and to 64 bits for x86-64: $\text{FFFFFFFF FFFFFFFCA}_{16}$

b. 3855

$$3855_{10} = 1111\ 0000\ 1111_2 (2048+1024+512+256+8+4+2+1)$$

$$1111\ 0000\ 1111_2 = \text{F0F}_{16}$$

expand to 32 bits for IA32 and Sun: 00000F0F_{16} ,

and to 64 bits for x86-64: $00000000\ 00000\text{F0F}_{16}$

c. -6691

$$-6691_{10} = 10\ 0101\ 1101\ 1101_2 (-8192 + (1024+256+128+64+16+8+4+1))$$

(Expand to 16 bit before conversion): $1110\ 0101\ 1101\ 1101_2 = \text{E5DD}_{16}$

expand to 32 bits for IA32 and Sun: FFFFE5DD_{16} ,

and to 64 bits for x86-64: $\text{FFFFFFFF FFFFE5DD}_{16}$

-862			3855			-6691		
IA32	x86-64	Sun	IA32	x86-64	Sun	IA32	x86-64	Sun
A2	A2	FF	0F	0F	00	DD	DD	FF
FC	FC	FF	0F	0F	00	E5	E5	FF
FF	FF	FC	00	00	0F	FF	FF	E5
FF	FF	A2	00	00	0F	FF	FF	DD
	FF			00			FF	
	FF			00			FF	
	FF			00			FF	

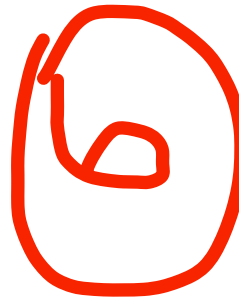
IA32, x86-64, and Sun representations

12

4. [6 points] Translate the following unsigned binary numbers into their base 10 equivalents. Use fractions in lowest terms to represent values on the right-hand side of the binary point.

a. $0.0101011 = \frac{43}{128}$
 $\frac{1}{4} + \frac{1}{16} + \frac{1}{64} + \frac{1}{128} = \frac{32+8+2+1}{128} = \frac{43}{128}$

b. $11100.0101 = 28\frac{5}{16}$
 $16 + 8 + 4 + \frac{1}{4} + \frac{1}{16} = 28\frac{5}{16}$



5. [6 points] Translate the following decimal numbers into their binary equivalents using a binary point.

a. $79\frac{1}{2}$

$$79\frac{1}{2} = 64 + 8 + 4 + 2 + 1 + \frac{1}{2} = 1001111.1_2$$



b. $62\frac{9}{16}$

$$62\frac{9}{16} = 32 + 16 + 8 + 4 + 2 + \frac{1}{2} + \frac{1}{16} = 111110.1001_2$$



6

6. [20 points] Consider the following C code. Assume that variables are stored in memory starting at location 1000, in order and with no gaps between variables. Assume all int's and pointers consume 4 bytes each.

- Show the memory layout and values for the variables at position A.
- Show the memory layout and values for the variables at position B.
- Show the memory layout and values for the variables at position C.
- What is printed by the program?

```
#include <stdio.h>
void main ()
{
    int x = 3, y = 0;
    int *p = &x;
    int *q = &y;
    int a[3] = {-1, 2, 6};
    /* position A */
    *q += 1;
    q--;
    *q += 2;
    /* position B */
    q = &a[1];
    *q = 7;
    p = q++;
    *p = 4;
    *q = 5;
    /* position C */
    printf ("%d, %d, %d\n", &p, p, *p);
    printf ("%d, %d, %d\n", &q, q, *q);
}
```

10

a-c.

variable	address	value at A	value at B	value at C
x	1000	3	5	5
y	1004	0	1	1
*p	1008	1000	1000	1020
*q	1012	1004	1000	1024
a[0]	1016	-1	-1	-1
a[1]	1020	2	2	4
a[2]	1024	6	6	5

d. Program prints:

1008, 1020, 4

1012, 1024, 5

✓

✓